4 2D Convolution (10 pts)

Problem 4.1 (5 pts)

In [2]:
```
import numpy as np
```

In [3]:
```
matrix = np.array([
    [0,  0, -1,  0, 0, 0, 1, 0, 0],
    [0, -1, -1, -1, 0, 1, 1, 1, 0],
    [-1, -1, -1, -1, 0, 1, 1, 1, 1],
    [0, -1, -1, -1, 0, 1, 1, 1, 0],
    [0,  0, -1,  0, 0, 0, 1, 0, 0]
])

kernel = np.array([
    [0, -1/2, 0],
    [-1/2, 2, -1/2],
    [0, -1/2, 0]
])

print(matrix)
print(kernel)

print("Shape:", matrix.shape)  # Outputs: (5, 9)
print("Shape: ", kernel.shape) # Outputs: (3, 3)
```

```
[[ 0  0 -1  0  0  0  1  0  0]
 [ 0 -1 -1 -1  0  1  1  1  0]
 [-1 -1 -1 -1  0  1  1  1  1]
 [ 0 -1 -1 -1  0  1  1  1  0]
 [ 0  0 -1  0  0  0  1  0  0]]
[[ 0.  -0.5  0. ]
 [-0.5  2.  -0.5]
 [ 0.  -0.5  0. ]]
Shape: (5, 9)
Shape:  (3, 3)
```

In [4]:
```
padded_matrix = np.pad(matrix, pad_width=1, mode='constant', constant_values=0)
padded_matrix
```

Out[4]:
```
array([[ 0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0],
       [ 0,  0,  0, -1,  0,  0,  0,  1,  0,  0,  0],
       [ 0,  0, -1, -1, -1,  0,  1,  1,  1,  0,  0],
       [ 0, -1, -1, -1, -1,  0,  1,  1,  1,  1,  0],
       [ 0,  0, -1, -1, -1,  0,  1,  1,  1,  0,  0],
       [ 0,  0,  0, -1,  0,  0,  0,  1,  0,  0,  0],
       [ 0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0]])
```

In [5]:
```
# Single convolution multiplication

hadamard_result = padded_matrix[:3, :3] * kernel
pixel_output = np.sum(hadamard_result)

print("Hadamard Matrix: ", hadamard_result)
print("Final pixel value: ", pixel_output)
```

```
Hadamard Matrix:  [[ 0. -0.  0.]
 [-0.  0. -0.]
 [ 0. -0. -0.]]
Final pixel value:  0.0
```

In [6]:
```
output = np.zeros((5, 9))

for i in range(0, 5):
    for j in range(0, 9):
        image_patch = padded_matrix[i:i+3, j:j+3]
        output[i, j] = np.sum(image_patch * kernel)

print("Final Convolved Output: ")
print(output)
```

```
Final Convolved Output:
[[ 0.   1.  -1.5  1.   0.  -1.   1.5 -1.   0. ]
 [ 1.  -1.   0.  -1.   0.   1.   0.   1.  -1. ]
 [-1.5  0.   0.  -0.5  0.   0.5  0.   0.   1.5]
 [ 1.  -1.   0.  -1.   0.   1.   0.   1.  -1. ]
 [ 0.   1.  -1.5  1.   0.  -1.   1.5 -1.   0. ]]
```

Problem 4.2 (5 pts)

In [7]:
```python
import numpy as np
from scipy.signal import convolve2d
from PIL import Image
import matplotlib.pyplot as plt


# Image Source: https://unsplash.com/photos/the-interior-of-a-car-NkBFPN9TSnQ
img = Image.open("hw1_img_001.jpg").convert("L")
arr = np.array(img, dtype=float)

kernel = np.array([[0, -0.5, 0],
                   [-0.5, 2, -0.5],
                   [0, -0.5, 0]])

out = convolve2d(arr, kernel, mode="same", boundary="fill", fillvalue=0)
out_clipped = np.clip(out, 0, 255).astype(np.uint8)

# Display original and processed images side-by-side
fig, axes = plt.subplots(1, 2, figsize=(10, 5))

axes[0].imshow(arr, cmap='gray')
axes[0].set_title('Original Image')
axes[0].axis('off')

axes[1].imshow(out_clipped, cmap='gray')
axes[1].set_title('Edge Detected (Convolved)')
axes[1].axis('off')

plt.show()
```



5. Lab: LMS Algorithm (15 pts)

(a) (3pt) Directly compute the least square (Wiener) solution with the provided dataset. What is the optimal weight W∗? What is the MSE loss of the whole dataset when the weight is set to W∗?

In [8]:
```
import numpy as np
import scipy.io as sio

mat_contents = sio.loadmat('dataset.mat')
print(sorted(mat_contents.keys()))

my_D = mat_contents['D']
```

```
my_X = mat_contents['X']

W = np.linalg.solve(my_X.T @ my_X, my_X.T @ my_D)

K = my_X.shape[0]
y = my_X @ W
e = my_D - y
J = (1/(2*K)) * (e.T @ e)

print("Optimal W: ", W)
print("MSE Loss: ", J.item())
```

```
['D', 'X', '__globals__', '__header__', '__version__']
Optimal W:  [[ 1.0006781 ]
 [ 1.00061145]
 [-2.00031968]]
MSE Loss:  5.0399515658683515e-05
```

(b) (4pt) Now consider that you can only train with 1 pair of data points and target each time. In such a case, the LMS algorithm should be used to find the optimal weight. Please initialize the weight vector as W0 = [−0.2, 0.0, 0.45]T , and update the weight with the LMS algorithm. After each epoch (every time you go through all the training data and loop back to the beginning), compute and record the MSE loss of the current weight on the whole dataset. Run LMS for 20 epochs with learning rate r = 0.005, report the weight you get in the end, and plot the MSE loss in log scale vs. Epochs.

In [9]:
```
import matplotlib.pyplot as plt

W0 = np.array([-0.2, 0.0, 0.45]).reshape(-1, 1)
current_W = W0.copy()

r = 0.005
K = my_X.shape[0]
J_history = []

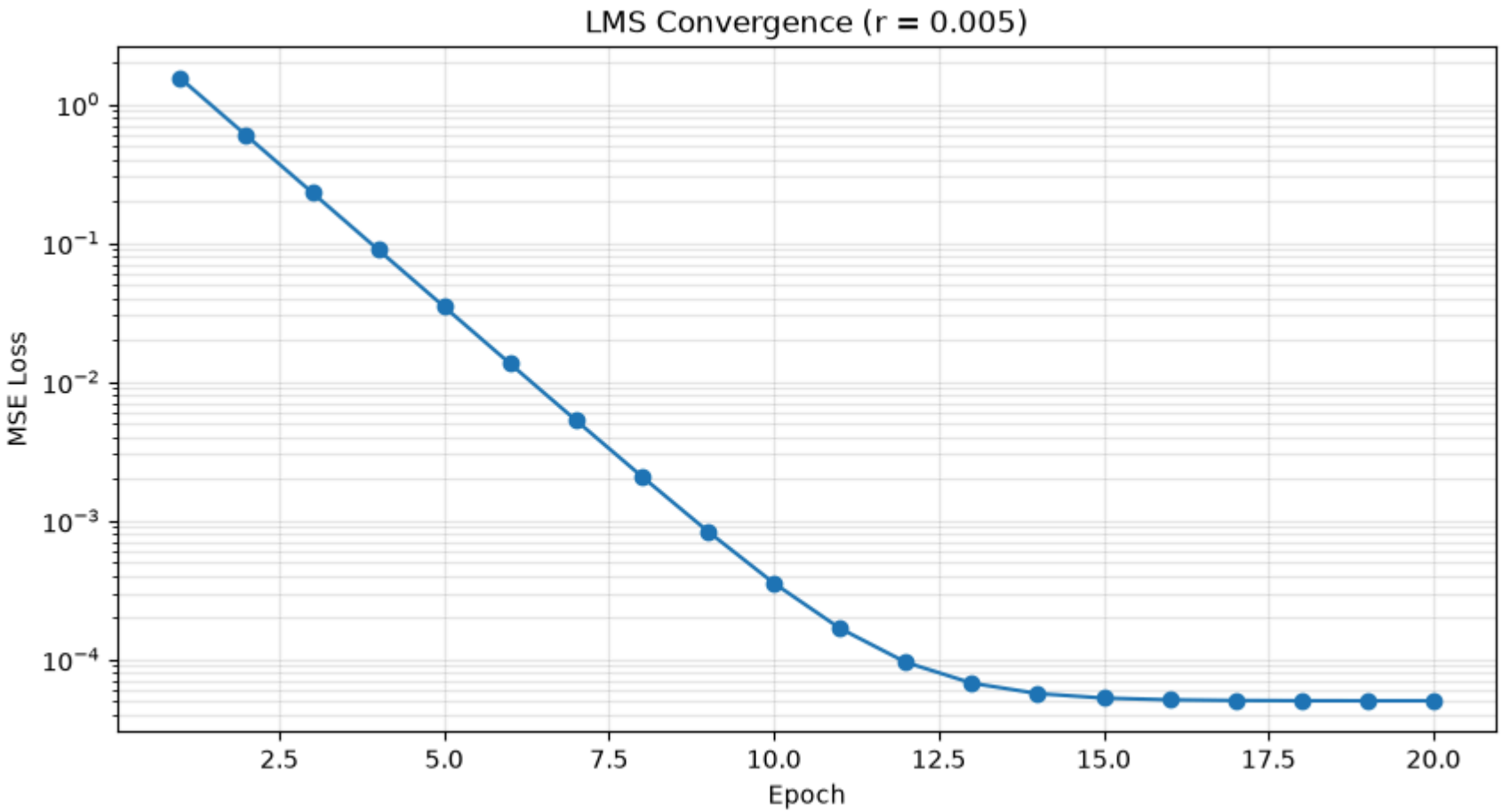
for epoch in range(20):
    for k in range(K):
        x_k = my_X[k:k+1, :].T
        d_k = my_D[k, 0]
        e_k = d_k - (x_k.T @ current_W).item()
        current_W = current_W + r * e_k * x_k
```

```
    y = my_X @ current_W
    e = my_D - y
    J = (1/(2*K)) * (e.T @ e)
    J_history.append(J.item())

print("Final W after 20 epochs:", current_W)

plt.figure(figsize=(10, 5))
plt.plot(range(1, 21), J_history, marker='o')
plt.yscale('log')
plt.xlabel('Epoch')
plt.ylabel('MSE Loss')
plt.title('LMS Convergence (r = 0.005)')
plt.grid(True, which='both', alpha=0.3)
plt.show()
```

```
Final W after 20 epochs: [[ 1.00066744]
 [ 1.0005943 ]
 [-2.00030134]]
```



(3pt) Scatter plot the points (x1k, x2k, dk) for all 100 data-target pairs in a 3D figureb, and plot the lines corresponding to the linear models you got in (a) and (b) respectively in the same figure. Observe if the linear models fit the data well.

In [10]:
```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
from matplotlib.lines import Line2D


x1 = my_X[:, 1]
x2 = my_X[:, 2]
```

```
d  = my_D[:, 0]

N = 25
x1_range = np.linspace(x1.min(), x1.max(), N)
x2_range = np.linspace(x2.min(), x2.max(), N)
X1_grid, X2_grid = np.meshgrid(x1_range, x2_range)

w0_a, w1_a, w2_a = W.flatten()            # part (a) weight
D_plane_a = w0_a + w1_a * X1_grid + w2_a * X2_grid

w0_b, w1_b, w2_b = current_W.flatten()    # part (b) weight
D_plane_b = w0_b + w1_b * X1_grid + w2_b * X2_grid

fig = plt.figure(figsize=(10, 8))
ax = fig.add_subplot(111, projection='3d')

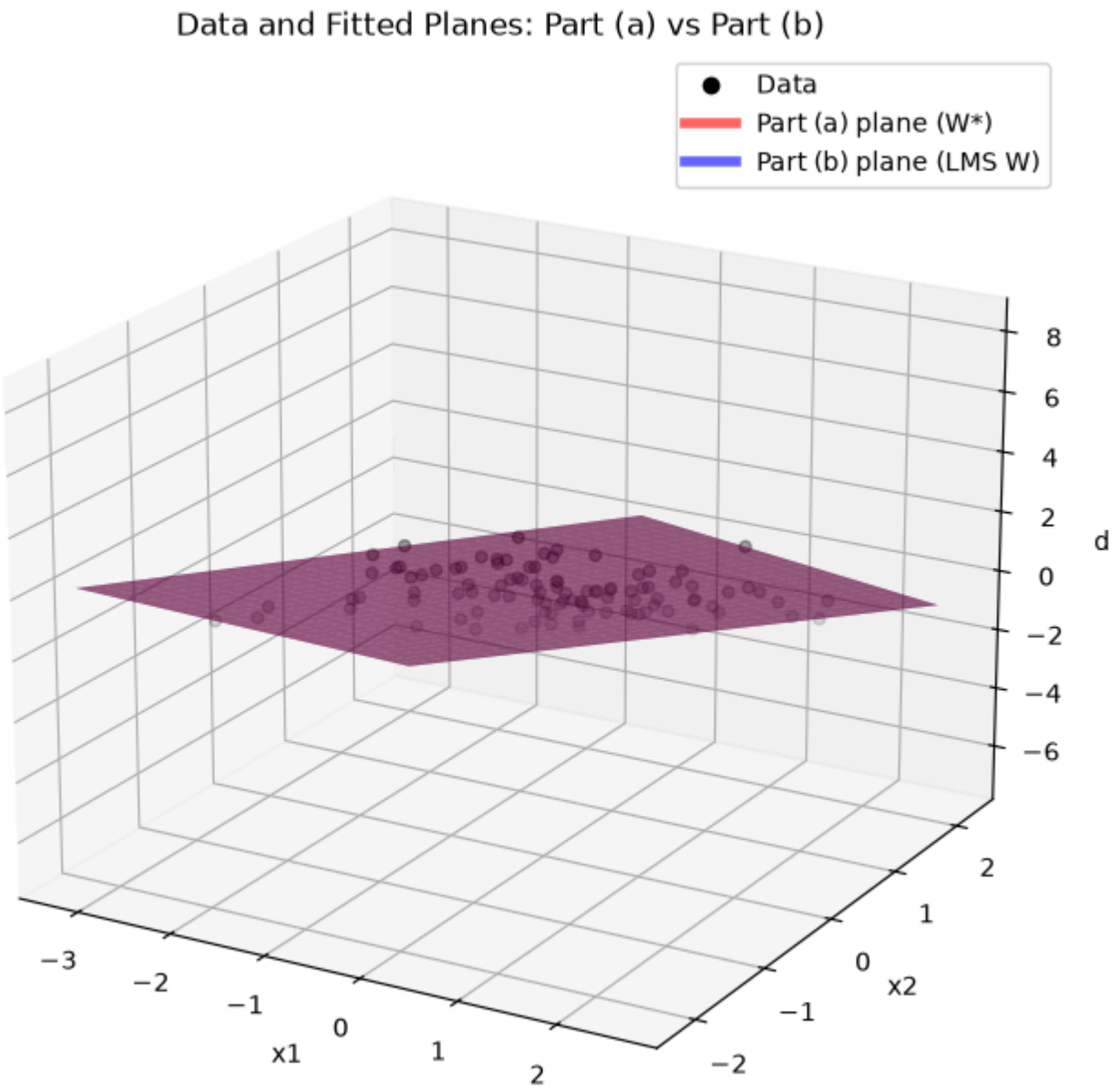
ax.scatter(x1, x2, d, color='black', alpha=0.6)
ax.plot_surface(X1_grid, X2_grid, D_plane_a, alpha=0.4, color='red')
ax.plot_surface(X1_grid, X2_grid, D_plane_b, alpha=0.4, color='blue')

ax.view_init(elev=20, azim=-60)

ax.set_xlabel('x1')
ax.set_ylabel('x2')
ax.set_zlabel('d')
ax.set_title('Data and Fitted Planes: Part (a) vs Part (b)')

legend_elements = [
    Line2D([0], [0], marker='o', color='w', markerfacecolor='black', markersize=8, label='Data'),
    Line2D([0], [0], color='red', lw=4, alpha=0.6, label='Part (a) plane (W*)'),
    Line2D([0], [0], color='blue', lw=4, alpha=0.6, label='Part (b) plane (LMS W)'),
]
ax.legend(handles=legend_elements)

plt.show()
```


Data and Fitted Planes: Part (a) vs Part (b)
Data
Part (a) plane (W*)
Part (b) plane (LMS W)
8
6
4
2
0
−2
−4
−6
d
−3
−2
−1
0
1
2
x1
2
1
0
−1
−2
x2

(d) (5pt) Learning rate r is an important hyperparameter for the LMS algorithm, as well as for CNN optimization. Here, try repeat the process in (b) with r set to 0.01, 0.05, 0.1 and 0.5 respectively. Together with the result you got in (b), plot the MSE losses of all sets of experiments in log scale vs. Epochs in one figure. Then try further enlarge the learning rate to r = 1 and observe how the MSE changes. Based on these observations, comments on how the learning rate affects the speed and quality of the learning process. (Note: The learning rate tuning for the CNN optimization will be introduced in Lecture 7.)

In [11]:
```
def learning_rate_weight_update(W_init, r):
    current_W = W_init.copy()
    J_history = []
    K = my_X.shape[0]

    for epoch in range(20):
        for k in range(K):
            x_k = my_X[k:k+1, :].T
            d_k = my_D[k, 0]
            e_k = d_k - (x_k.T @ current_W).item()
            current_W = current_W + r * e_k * x_k

        y = my_X @ current_W
        e = my_D - y
        J = (1/(2*K)) * (e.T @ e)
        J_history.append(J.item())

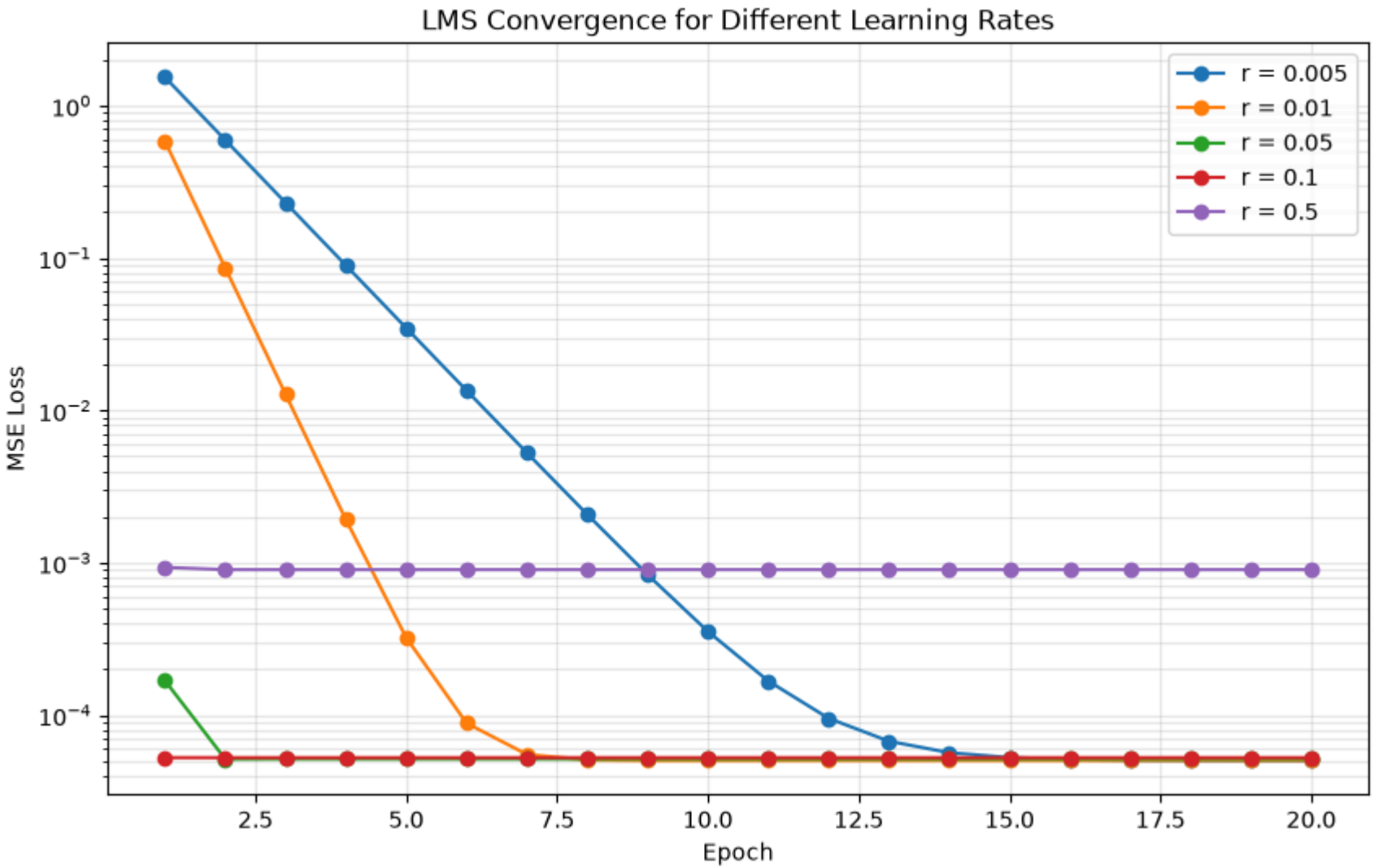
    return J_history, current_W


W0 = np.array([-0.2, 0.0, 0.45]).reshape(-1, 1)
lst_learning_rate = [0.005, 0.01, 0.05, 0.1, 0.5]

results = {}   # r -> (J_history, final_W)
for r in lst_learning_rate:
    J_hist, final_W = learning_rate_weight_update(W0, r)
    results[r] = (J_hist, final_W)

plt.figure(figsize=(10, 6))
for r in lst_learning_rate:
    J_hist, _ = results[r]
    plt.plot(range(1, 21), J_hist, marker='o', label=f'r = {r}')
```

```
plt.yscale('log')
plt.xlabel('Epoch')
plt.ylabel('MSE Loss')
plt.title('LMS Convergence for Different Learning Rates')
plt.legend()
plt.grid(True, which='both', alpha=0.3)
plt.show()
```



In [12]:
```
J_hist_r1, final_W_r1 = learning_rate_weight_update(W0, 1)

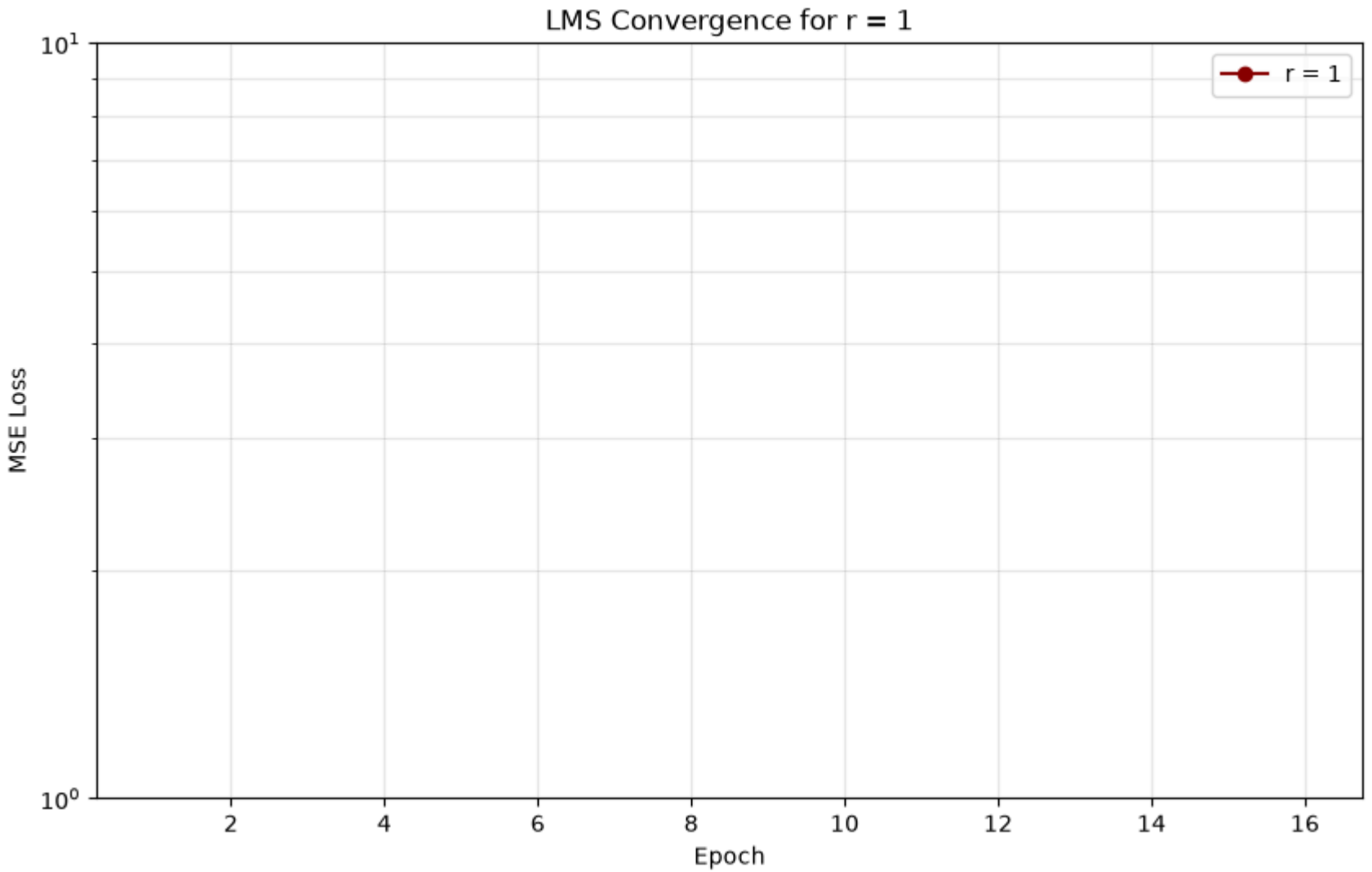
plt.figure(figsize=(10, 6))
plt.plot(range(1, 21), J_hist_r1, marker='o', color='darkred', label='r = 1')

plt.yscale('log')
plt.xlabel('Epoch')
```

```
plt.ylabel('MSE Loss')
plt.title('LMS Convergence for r = 1')
plt.legend()
plt.grid(True, which='both', alpha=0.3)
plt.show()

print("Final W after 20 epochs (r = 1):", final_W_r1)
print("J history (r = 1):", J_hist_r1)
```

```
/Users/temesmacbookair/Development/ece661_ce_ml_nn/.venv/lib/python3.14/site-packages/matplotlib/scale.py:375: Runti
meWarning: overflow encountered in exp
  return np.exp(values * np.log(self.base))
```




```
Final W after 20 epochs (r = 1): [[ 1.28471518e+188]
 [-4.10303918e+187]
 [-1.33708699e+188]]
J history (r = 1): [6.330673253512807e+18, 4.1084066028668144e+37, 2.6662243288047043e+56, 1.73029421346673e+75, 1.1
22905538296755e+94, 7.287297374769916e+112, 4.729222647603912e+131, 3.0691140625114774e+150, 1.991756749595622e+16
9, 1.292586351878208e+188, 8.388471520937628e+206, 5.443849407456229e+225, 3.53288394638893e+244, 2.292728553724759e
+263, 1.4879074152542656e+282, 9.656042677934965e+300, inf, inf, inf, inf]
```

Answer: A bigger learning rate makes the model learn faster. 0.05 and 0.1 reach their best result in just one pass through the data, but a small rate like 0.005 takes over 15 passes to get there. But if the rate gets too big, like 0.5, the model starts overshooting the target slightly each time, so it ends up a bit less accurate even though it's fast. At r=1, the overshooting becomes so extreme that the model gets worse and worse instead of better.